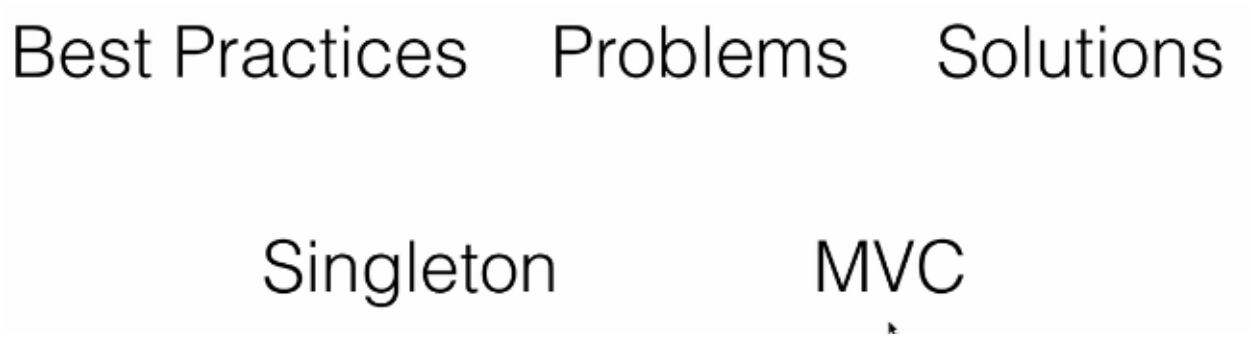
What is design pattern:

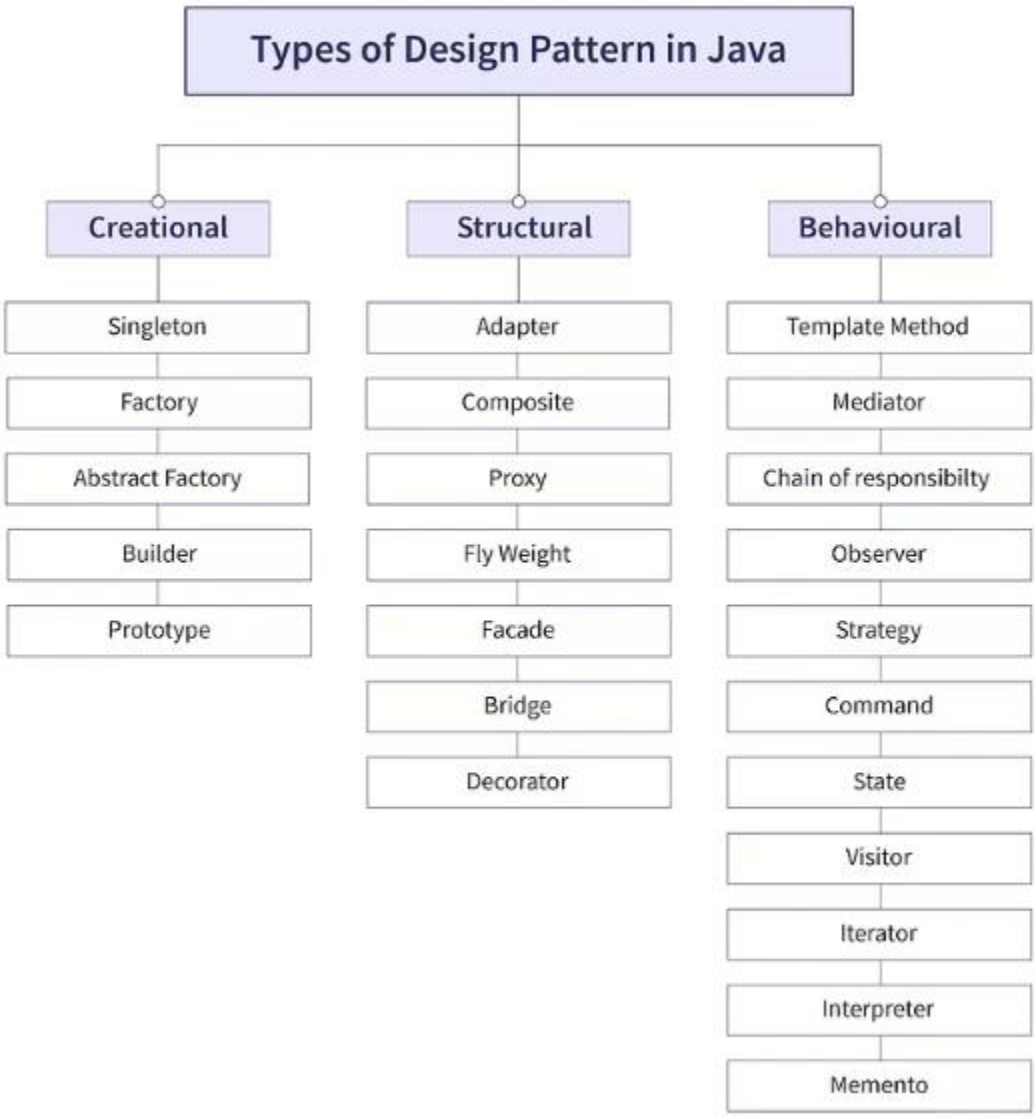
In software engineering, a **Design Pattern** is a tried-and-tested solution to a common problem that occurs during software design. Think of them as **blueprints** or templates that you can customize to solve a specific architectural challenge in your code.

Design patterns in Java refer to structured approaches involving objects and classes that aim to solve recurring design issues within specific contexts. These patterns offer reusable, general solutions to common problems encountered in software development, representing established best practices. By utilizing design patterns, developers can communicate more effectively about their approaches to problem-solving, fostering collaboration and consistency in their code.



Why Use Design Patterns?

- **Standardization:** They provide a common vocabulary for developers. Instead of explaining a complex hidden logic, you can simply say, "I used a Singleton here."
- **Best Practices:** They represent the "best of" solutions discovered by experienced developers over decades.
- **Code Stability:** By following proven patterns, you reduce the risk of architectural errors that could force a total rewrite later.



Creational Design Patterns:

Creational design patterns software engineering mein wo patterns hote hain jo **object creation mechanisms** (object banane ke tareeke) se deal karte hain.

Creational design patterns provide various object creation mechanisms, which increase flexibility and reuse of existing code.

Creational Design Patterns Kyun Use Karein?

- **Complexity chhupane ke liye:** Object kaise ban raha hai, uski complexity user se chhupi rehti hai.
- **Flexibility badhane ke liye:** Aap decide kar sakte hain ki kab, kaunsa, aur kaise object create karna hai bina main logic ko chede.
- **Control ke liye:** Jaise ki agar aap chahte hain ki poori application mein kisi class ka sirf **ek hi** object bane.

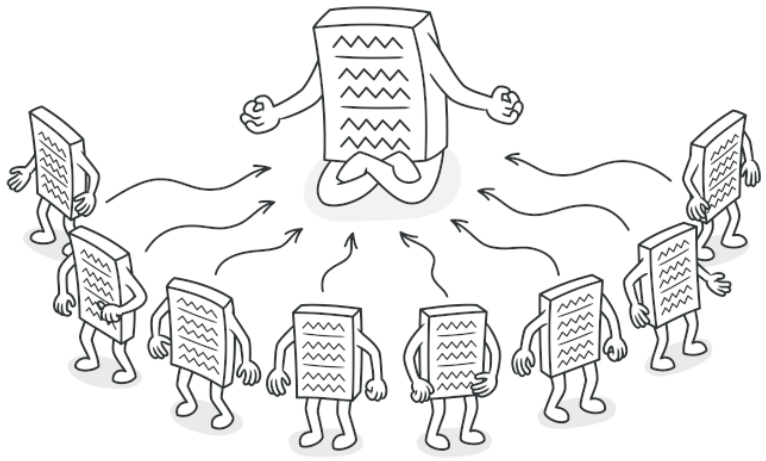
Creational Design Patterns

1. **Singleton:** Yeh pakka karta hai ki class ka sirf **ek hi object** bane aur sab wahi use karein. Database connections ke liye best hai.
2. **Factory Method:** Object banane ka kaam ek **special method** ko de deta hai, taaki client ko ye na pata chale ki kaunsi class ka object ban raha hai.
3. **Abstract Factory:** Yeh related objects ki poori **family (group)** ek saath banane mein madad karta hai (Jaise Dark Theme ke saare buttons aur icons).
4. **Builder:** Complex objects ko **step-by-step** banata hai. Agar object banane ke liye bahut saari details chahiye (jaise Pizza toppings), toh ye code ko saaf rakhta hai.
5. **Prototype:** Naya object zero se banane ke bajaye **purane object ki copy (clone)** bana deta hai. Yeh memory aur time bachane ke liye use hota hai.

1. Singleton Pattern:

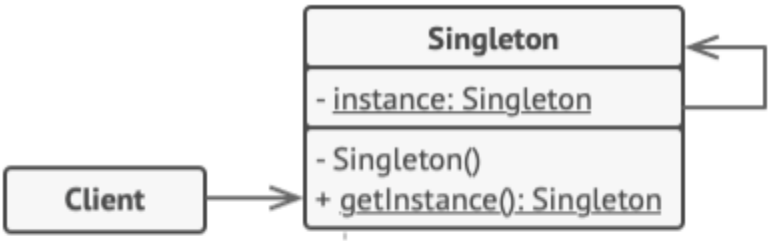
Iska maqsad ye ensure karna hota hai ki ek class ka poori application mein sirf **ek hi instance (object)** ho. Ye aksar Database connection ya Loggers ke liye use hota hai.

Singleton is a creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance.



The **Singleton** class declares the static method getInstance that returns the same instance of its own class.

The Singleton's constructor should be hidden from the client code. Calling the getInstance method should be the only way of getting the Singleton object.



The **Singleton** class declares the static method `getInstance` that returns the same instance of its own class.

The Singleton's constructor should be hidden from the client code. Calling the `getInstance` method should be the only way of getting the Singleton object.

```
if (instance == null) {
   // Note: if you're creating an app with
   // multithreading support, you should
   // place a thread lock here.
   instance = new Singleton()
}
return instance
```

Java mein ek perfect Singleton class banane ke liye 3 cheezein zaroori hain:

1. **Private Constructor:** Taaki bahar se koi `new` keyword use karke naya object na bana sake.
2. **Private Static Variable:** Jo us single instance ko hold karega.
3. **Public Static Method:** Jo us instance ko return karega (ise aksar `getInstance()` kehte hain).

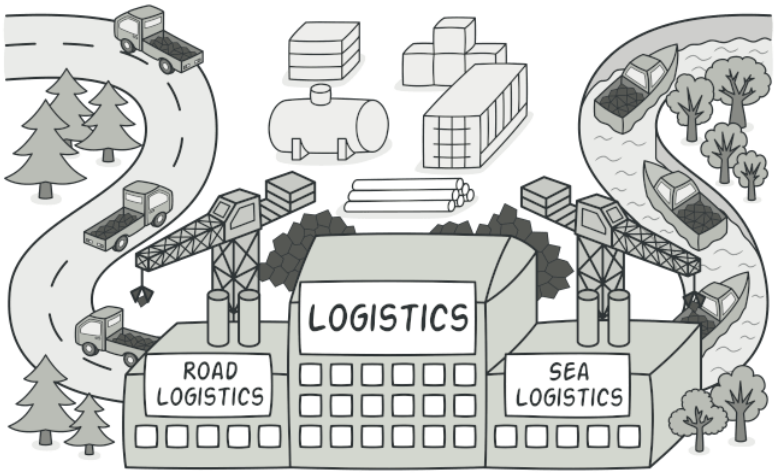
```
public class AppConfig {

    // 1. Static variable jo class ka ek hi object store karega
    private static AppConfig instance;

    public String appName = "My Awesome App";

    // 2. Private Constructor: Taaki bahar se 'new AppConfig()' na ho sake
    private AppConfig() { }

    // 3. Static method: Jo object ko return karega
    public static AppConfig getInstance() {
        if (instance == null) {
            instance = new AppConfig(); // Pehli baar call hone par object banega
        }
        return instance; // Wahi purana object return hoga
    }
}
```

2. Factory Method Pattern:

Factory Method Pattern ek aisa creational design pattern hai jo ek interface ya class banata hai objects create karne ke liye, lekin sub-classes ko allow karta hai ki wo decide karein ki kis class ka object instantiate (banana) hai.

Iska sabse bada fayda ye hai ki aapka main code (client code) specific classes se "tightly coupled" nahi hota.

Factory Method is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.



Factory Method ka Simple Example: **Shape Factory:**

```
public interface Shape {
    void draw();
}
```

```
class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing a Circle!");
    }
}

class Square implements Shape {
    public void draw() {
        System.out.println("Drawing a Square!");
    }
}
```

3. Factory Class (The Creator)

Ye class object banane ka decision legi.

Java

```
public class ShapeFactory {
    // Ye method decide karega kaunsa object banana hai
    public Shape getShape(String shapeType) {
        if (shapeType == null) return null;

        if (shapeType.equalsIgnoreCase("CIRCLE")) {
            return new Circle();
        } else if (shapeType.equalsIgnoreCase("SQUARE")) {
            return new Square();
        }
        return null;
    }
}
```

4. Client Code (Isko Use Kaise Karein)

Ab user ko ye tension lene ki zaroorat nahi ki `Circle` kaise banta hai ya `Square` kaise. Wo bas Factory se shape ka naam bolega.

Java

```
public class Main {
    public static void main(String[] args) {
        ShapeFactory factory = new ShapeFactory();

        // Factory se Circle maanga
        Shape s1 = factory.getShape("CIRCLE");
        s1.draw();

        // Factory se Square maanga
        Shape s2 = factory.getShape("SQUARE");
        s2.draw();
    }
}
```

Factory Method Kyun Use Karein? (Benefits)

- **Loose Coupling:** Client code ko nahi pata ki actual classes (`Circle` , `Square`) ka naam kya hai ya wo kaise kaam karti hain.
- **Scalability:** Agar kal ko "Triangle" shape add karni hai, toh aapko bas ek nayi class banani hai aur Factory mein ek `else if` dalna hai. Main code waisa hi rahega.
- **Single Responsibility:** Object banane ka logic ek hi jagah (Factory mein) hota hai.

Singleton vs Factory Method

- **Singleton:** Ek class ka **sirf ek** object banane ke liye.
- **Factory Method:** Kai classes mein se **kaunsa** object banana hai, ye handle karne ke liye.

3. Abstract Factory Pattern:

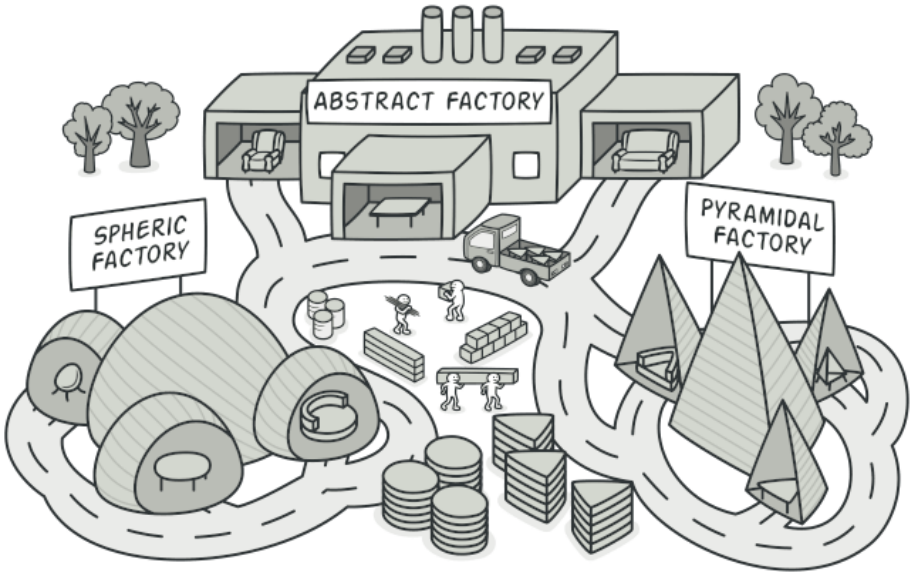
Abstract Factory Pattern ko "Factory of Factories" bhi kaha jata hai.

Jab aapke paas bahut saare related objects hote hain aur aap unhe unki "families" ke hisaab se banana chahte hain, tab Abstract Factory ka use hota hai.

The **Abstract Factory Pattern** is a creational design pattern that allows you to produce families of related objects without specifying their concrete classes.

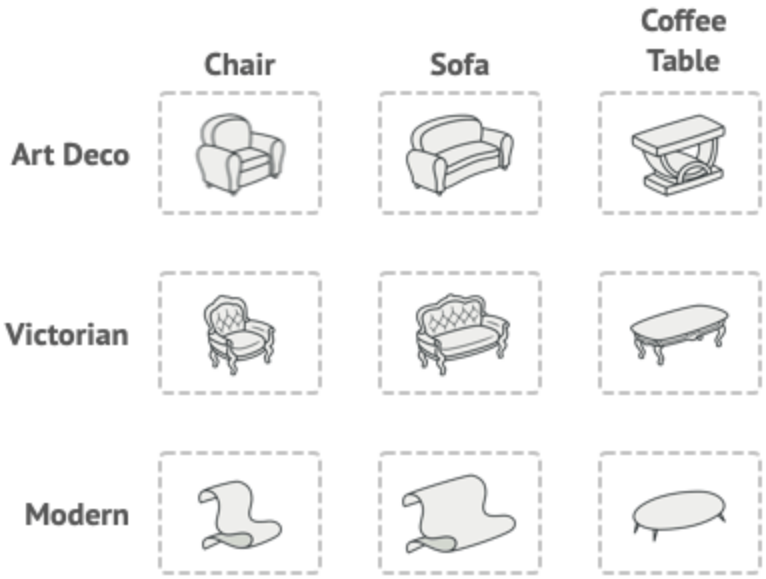
Think of it as a "factory of factories." It provides a way to encapsulate a group of individual factories that have a common theme.

Abstract Factory ek **"Factory of Factories"** hai.


ABSTRACT FACTORY
SPHERIC FACTORY
PYRAMIDAL FACTORY

Imagine that you're creating a furniture shop simulator. Your code consists of classes that represent:

1. A family of related products, say: `Chair` + `Sofa` + `CoffeeTable`.
2. Several variants of this family. For example, products `Chair` + `Sofa` + `CoffeeTable` are available in these variants: `Modern`, `Victorian`, `ArtDeco`.



Product families and their variants.

Bilkul, wahi Furniture wala example detail mein dekhte hain. Is pattern ka asli maza tab aata hai jab aapke paas multiple product categories (Chair, Sofa, Table) aur multiple styles (Modern, Victorian, Rustic) honge.

Ye example **Abstract Factory Pattern** ko perfectly justify karta hai kyunki yahan **"Consistency"** sabse zyada zaroori hai.

1. The Products (Interfaces)

Pehle hum har product ki ek generic identity (interface) banayenge.

Java

```
interface Chair {
    void hasLegs();
    void sitOn();
}

interface Sofa {
    void isComfortable();
}

interface CoffeeTable {
    void putCoffee();
}
```

2. Concrete Products (Modern vs Victorian)

Ab har style ke liye alag classes banengi.

Modern Style:

Java

```
class ModernChair implements Chair {
    public void hasLegs() { System.out.println("Modern Chair has 4 steel legs.");
    public void sitOn() { System.out.println("Sitting on a Modern Chair."); }
}

class ModernSofa implements Sofa {
    public void isComfortable() { System.out.println("Modern Sofa is very sleek.'
}
```

Victorian Style:

Java

```
class VictorianChair implements Chair {
    public void hasLegs() { System.out.println("Victorian Chair has wooden carved
    public void sitOn() { System.out.println("Sitting on a Royal Victorian Chair.
}

class VictorianSofa implements Sofa {
    public void isComfortable() { System.out.println("Victorian Sofa is heavy and
}
```

3. The Abstract Factory

Ye interface decide karega ki ek "Furniture Set" mein kya kya hona chahiye.

Java

```
interface FurnitureFactory {
    Chair createChair();
    Sofa createSofa();
    CoffeeTable createCoffeeTable();
}
```

4. Concrete Factories (The Style Experts)

Har factory ek specific "Family" ka dhyan rakhegi.

Java

```
// Sirf Modern furniture banayegi
class ModernFurnitureFactory implements FurnitureFactory {
    public Chair createChair() { return new ModernChair(); }
    public Sofa createSofa() { return new ModernSofa(); }
    public CoffeeTable createCoffeeTable() { return new ModernCoffeeTable(); }
}

// Sirf Victorian furniture banayegi
class VictorianFurnitureFactory implements FurnitureFactory {
    public Chair createChair() { return new VictorianChair(); }
    public Sofa createSofa() { return new VictorianSofa(); }
    public CoffeeTable createCoffeeTable() { return new VictorianCoffeeTable(); }
}
```

5. Client Code (The Shop)

Client ko bas ye batana hai ki use kaunsa "Theme" chahiye.

Java

```
public class FurnitureShop {
    private Chair chair;
    private Sofa sofa;

    // Client ko nahi pata chair modern hai ya victorian
    // Use bas "Chair" interface se matlab hai
    public FurnitureShop(FurnitureFactory factory) {
        chair = factory.createChair();
        sofa = factory.createSofa();
    }

    public void showOffFurniture() {
        chair.sitOn();
        sofa.isComfortable();
    }
```

```
    public static void main(String[] args) {
        // Customer wants Modern theme
        FurnitureFactory modernFactory = new ModernFurnitureFactory();
        FurnitureShop shop1 = new FurnitureShop(modernFactory);
        shop1.showOffFurniture();

        // Customer wants Victorian theme
        FurnitureFactory victorianFactory = new VictorianFurnitureFactory();
        FurnitureShop shop2 = new FurnitureShop(victorianFactory);
        shop2.showOffFurniture();
    }
}
```

4. Builder Pattern:

Jab ek object bahut complex hota hai aur usse banane ke liye bahut saare steps chahiye hote hain, tab Builder pattern ka use hota hai. Ye step-by-step object construction mein help karta hai.

Builder Pattern Kya Hai

Maan lo aapko ek **Pizza** order karna hai. Pizza mein bahut saari complexity ho sakti hai:

- Size (Small, Medium, Large)
- Cheese (Extra cheese, No cheese)
- Toppings (Paneer, Olives, Mushroom, Pepperoni)
- Crust (Thin crust, Pan pizza)

Agar aap ek hi `Pizza` class ka **Constructor** banayenge, toh woh kuch aisa dikhega: `Pizza myPizza = new Pizza("Medium", true, true, false, true, "Thin Crust");`

Yeh **"Telescoping Constructor"** problem hai—aapko yaad hi nahi rahega ki kaunsa `true` cheese ke liye hai aur kaunsa toppings ke liye!

Builder Pattern is complex process ko step-by-step tod deta hai.

1. The Product (Pizza)

Java

```java
public class Pizza {
    private String size;
    private boolean cheese;
    private boolean pepperoni;
    private String crust;

    // Setters (usually called by Builder)
    public void setSize(String size) { this.size = size; }
    public void setCheese(boolean cheese) { this.cheese = cheese; }
    public void setPepperoni(boolean pepperoni) { this.pepperoni = pepperoni; }
    public void setCrust(String crust) { this.crust = crust; }

    public void show() {
        System.out.println("Pizza [Size=" + size + ", Cheese=" + cheese + ", Crus
    }
}
```

2. The Builder (Step-by-step maker)

Java mein hum aksar **Static Inner Class** use karte hain builder ke liye:

Java

```java
public class PizzaBuilder {
    private Pizza pizza = new Pizza();

    public PizzaBuilder setSize(String size) {
        pizza.setSize(size);
        return this; // "this" return karne se method chaining hoti hai
    }

    public PizzaBuilder addCheese() {
        pizza.setCheese(true);
        return this;
    }

    public PizzaBuilder setCrust(String crust) {
        pizza.setCrust(crust);
        return this;
    }

    public Pizza build() {
        return this.pizza; // Final object return karta hai
    }
```

3. Client Code (Ordering Pizza)

Ab dekho code kitna saaf (readable) lagta hai:

Java

```java
public class Main {
    public static void main(String[] args) {
        // Method Chaining ka maza
        Pizza myPizza = new PizzaBuilder()
                            .setSize("Large")
                            .setCrust("Thin Crust")
                            .addCheese()
                            .build();

        myPizza.show();
    }
}
```

5. Prototype Pattern:

Agar ek object banana bahut "expensive" (memory ya time consuming) hai, toh hum naya object banane ki jagah purane object ki **copy (clone)** bana lete hain.

Maan lo aapke paas ek aisa object hai jise create karne mein bahut mehnat lagti hai—shayad woh database se heavy data fetch karta hai ya koi complex calculation karta hai.

Ab agar aapko bilkul waisa hi ek aur object chahiye, toh aap fir se wahi heavy process follow nahi karna chahenge. **Prototype Pattern** kehta hai: "Naya object scratch se banane ke bajaye, purane object ki **Copy (Clone)** bana lo."

Isse hum "Object Cloning" bhi kehte hain.

Key Concept: The `Cloneable` Interface

Java mein is pattern ko implement karne ke liye hum `Cloneable` interface aur `clone()` method ka use karte hain.

Java Code Example: Shape System

Socho aap ek Drawing App bana rahe ho jahan user "Circle" ko copy-paste kar sakta hai.

1. The Prototype Interface (Blueprint)

Java

```java
// Java mein hum Cloneable use karte hain
abstract class Shape implements Cloneable {
    private String id;
    protected String type;

    abstract void draw();

    public String getType() { return type; }
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    // Yeh hai asli magic method
    public Object clone() {
        Object clone = null;
        try {
            clone = super.clone();
        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
        }
        return clone;
```

2. Concrete Classes (Actual Objects)

Java

```java
class Circle extends Shape {
    public Circle() { type = "Circle"; }
    @Override
    public void draw() { System.out.println("Drawing a Circle."); }
}

class Rectangle extends Shape {
    public Rectangle() { type = "Rectangle"; }
    @Override
    public void draw() { System.out.println("Drawing a Rectangle."); }
}
```

3. Client Code (Cloning in Action)

Java

```
public class Main {
    public static void main(String[] args) {
        Circle circle1 = new Circle();
        circle1.setId("1");

        // Naya object banane ke bajaye, purane ko clone kiya
        Circle circle2 = (Circle) circle1.clone();

        System.out.println("Shape 1: " + circle1.getType());
        System.out.println("Shape 2 (Cloned): " + circle2.getType());
    }
}
```

Deep Copy vs Shallow Copy (Important Note)

Cloning karte waqt ek cheez ka dhyan rakhna padta hai:

- **Shallow Copy:** Sirf main object copy hota hai. Agar object ke andar koi aur object (reference) hai, toh dono (Original and Clone) ek hi memory location ko point karenge.
- **Deep Copy:** Poora object tree copy hota hai. Clone ekdum independent hota hai.

Structural Design Patterns:

Structural design patterns explain how to assemble objects and classes into larger structures, while keeping these structures flexible and efficient.

Structural Design Patterns ka main focus is baat par hota hai ki kaise classes aur objects ko aapas mein joda jaye (assemble kiya jaye) taaki ek bada aur flexible structure ban sake.

Agar Creational patterns "object banane" ke baare mein hain, toh Structural patterns **"objects ke rishte" (relationships)** ko manage karne ke baare mein hain. Inka maqsad ye hota hai ki agar system ka ek hissa change ho, toh poora structure disturb na ho.

Structural Patterns ka main kaam objects aur classes ko assemble karke ek bada aur flexible structure banana hota hai. Inka quick recap 5 points mein ye raha:

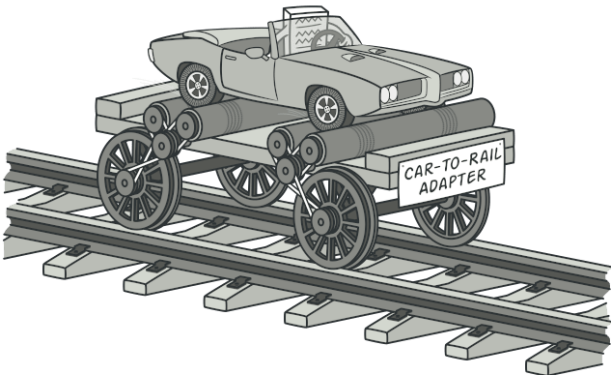
1. **Adapter (The Translator):** Do incompatible interfaces ko aapas mein jodta hai taki wo saath kaam kar saken (Jaise Indian plug ko USA socket mein lagana).
2. **Bridge (The Splitter):** Yeh "kya karna hai" (Abstraction) aur "kaise karna hai" (Implementation) ko alag-alag kar deta hai, taki dono ko independently badhaya ja sake.
3. **Composite (The Tree):** Objects ko tree structure mein arrange karta hai taki aap ek single item aur poore group (collection) ko ek hi tarah se treat kar saken (Jaise Files aur Folders).
4. **Decorator (The Wrapper):** Bina class change kiye, runtime par kisi object mein naye features ya behaviors add karta hai (Jaise Coffee mein extra toppings dalna).
5. **Facade (The Interface):** Ek bahut bade aur complex system ke upar ek simple "face" ya interface bana deta hai taki use use karna aasan ho jaye.

6. Proxy (The Representative)

- **Kaam:** Asli object ke liye ek "stand-in" ya placeholder ki tarah kaam karta hai.
- **Point:** Yeh access control (security) ya lazy loading (memory saving) ke liye use hota hai.
- **Example:** Internet access control karne wala Proxy Server ya heavy images ko tabhi load karna jab zaroorat ho.

1. Adapter Pattern:

Adapter is a structural design pattern that allows objects with incompatible interfaces to collaborate.



Adapter Pattern Kya Hai? (The "Jugaad" Pattern)

Iska real-world example hai ek **Power Adapter**. Maan lo aap India se USA gaye. India ke plugs 2-pin hote hain, par USA ke sockets 3-pin (flat) hote hain. Ab aap apna phone fenk toh nahi denge! Aap ek **Adapter** kharidte hain jo aapke Indian plug ko USA socket ke compatible bana deta hai.

Programming mein bhi yahi hota hai: Jab aapke paas ek **Existing Class** (Adaptee) hai jiska code aap badal nahi sakte, aur ek **Client** hai jo kisi specific interface (Target) ki demand kar raha hai, toh beech mein hum ek **Adapter** laga dete hain.

Stock market monitoring app ke liye **Adapter Pattern** ka example ekdum perfect hai.

Maan lo aapki app stock data dikhane ke liye ek popular **Analytics Library** use karti hai. Problem ye hai ki aapki app data ko `JSON` format mein handle karti hai, lekin wo library sirf `XML` format samajhti hai.

Dono incompatible hain, toh hum beech mein ek **Adapter** lagayenge jo JSON ko XML mein convert karke library ko de dega.

---

1. Target (Interface jo hamari App use karna chahti hai)

Hamari app ko ek simple interface chahiye stock data analyze karne ke liye.

Java

```java
interface StockProvider {
    void downloadAndDisplay(String jsonData);
}
```

2. Adaptee (Woh Library jiska interface match nahi kar raha)

Ye library bahut powerful hai, par ye sirf `XML` mangti hai. Hum iska source code change nahi kar sakte.

Java

```java
class ThirdPartyStockLibrary {
    public void analyzeStockData(String xmlData) {
        System.out.println("Analyzing Data from XML: " + xmlData);
        System.out.println("Showing Graphs and Trends...");
    }
}
```

3. The Adapter (JSON-to-XML Converter)

Yeh Adapter `StockProvider` ko implement karega aur `JSON` data ko `XML` mein convert karke library ko pass karega.

Java

```
class StockAdapter implements StockProvider {
    private ThirdPartyStockLibrary library;

    public StockAdapter() {
        this.library = new ThirdPartyStockLibrary();
    }

    @Override
    public void downloadAndDisplay(String jsonData) {
        // Step 1: JSON ko XML mein convert karo (Logic simplified for example)
        String xmlData = convertJsonToXml(jsonData);

        // Step 2: Library ka incompatible method call karo
        library.analyzeStockData(xmlData);
    }

    private String convertJsonToXml(String json) {
        return "<xml><stock>" + json + "</stock></xml>"; // Dummy Conversion
    }
```

4. Client Code (Hamari Mobile App)

App ko pata bhi nahi chalega ki peeche XML conversion ho raha hai.

Java

```
public class StockApp {
    public static void main(String[] args) {
        String myJsonData = "{ 'ticker': 'RELIANCE', 'price': 2500 }";

        // Hum Adapter use kar rahe hain jo interface ko match kar raha hai
        StockProvider provider = new StockAdapter();

        provider.downloadAndDisplay(myJsonData);
    }
}
```

2. Facade Pattern:

Facade is a structural design pattern that provides a simplified interface to a library, a framework, or any other complex set of classes.

Example: Jab aap Amazon par "Place Order" button dabate hain, toh peeche Inventory, Payment, aur Shipping jaise 10 complex systems kaam karte hain, lekin aapke liye sirf ek button (Facade) hota hai.

Facade Pattern Kya Hai? (The "Front Desk" Pattern)

Maan lo aap ek 5-star hotel mein gaye. Aapko room saaf karwana hai, khana order karna hai, aur laundry bhi deni hai.

- **Bina Facade ke:** Aapko Cleaning staff ko alag phone karna padega, Kitchen mein alag, aur Laundry wale ko alag.
- **With Facade (Receptionist):** Aap bas Receptionist ko phone karte ho, aur wo baaki sab handle kar leti hai.

Programming mein, jab ek system bahut complex ho jata hai (jisme 20-30 classes aapas mein judi hoti hain), toh hum ek **Facade Class** banate hain jo client ko sirf ek ya do simple methods deti hai.

Stock Market App Example (Facade)

Ek stock app mein jab aap **"Buy Stock"** par click karte ho, toh piche bahut saari cheezein hoti hain:

1. `PortfolioCheck` : Kya aapke paas balance hai?
2. `StockValidator` : Kya ye stock abhi trade ho raha hai?
3. `DatabaseLogger` : Transaction record karo.
4. `NotificationService` : SMS/Email bhejo.

Client (Mobile App) ke liye itni saari classes ko individually manage karna mushkil hai. Hum ek `StockFacade` banayenge jo ye sab handle karega.

1. Subsystem Classes (The Complex Stuff)

Java

```java
class Wallet {
    public boolean hasMoney() { return true; }
}

class Exchange {
    public void executeOrder() { System.out.println("Order executed on NSE/BSE!")
}

class Notification {
    public void sendSMS() { System.out.println("SMS Sent: Stock Purchased Succes
}
```

2. The Facade Class

Ye class saari complexity ko apne andar wrap kar leti hai aur client ko ek simple "face" (interface) deti hai.

Java

```java
class StockOrderFacade {
    private Wallet wallet = new Wallet();
    private Exchange exchange = new Exchange();
    private Notification notification = new Notification();

    public void quickBuy(String symbol) {
        System.out.println("Processing order for: " + symbol);
        if (wallet.hasMoney()) {
            exchange.executeOrder();
            notification.sendSMS();
            System.out.println("Done!");
        }
    }
}
```

3. Client Code

Ab client ko sirf ek hi method call karna hai.

Java

```java
public class Main {
    public static void main(String[] args) {
        // Client ko wallet ya exchange ke internal logic se matlab nahi hai
        StockOrderFacade stockApp = new StockOrderFacade();

        stockApp.quickBuy("TATASTEEL");
    }
}
```

Kab use karein?

- Jab aapka system bahut saari classes se milkar bana ho aur client ke liye use karna "sir dard" ban raha ho.
- Jab aap **Layers** banana chahte ho. Aap har layer ke liye ek Facade bana sakte ho jo entry point ki tarah kaam karega.

Summary: Adapter vs Facade

Aksar log inme confuse hote hain, par fark simple hai:

- **Adapter** tab use karo jab do cheezein **match nahi kar rahi** (Compatibility).
- **Facade** tab use karo jab system bahut **complex hai** aur use simple banana hai (Simplification).

3. Decorator Pattern:

Decorator is a structural design pattern that lets you attach new behaviors to objects by placing these objects inside special wrapper objects that contain the behaviors.

Ab aate hain **Decorator Pattern** par. Yeh pattern tab kaam aata hai jab aap kisi object ki functionality ko **dynamically** (runtime par) badhana chahte ho, bina uski original class ko chhede (modify kiye).

Decorator Pattern Kya Hai? (The "Toppings" Pattern)

Maan lo aap ek **Pizza** kha rahe ho.

1. Pehle aapne ek `BasePizza` liya.
2. Phir aapne us par `ExtraCheese` dalwaya.
3. Phir aapne `Mushroom` toppings dalwayi.

Yahan aap naya pizza nahi bana rahe, balki base pizza ko "decorate" kar rahe ho. Agar aap Inheritance use karte, toh aapko har combination ke liye alag class banani padti (`PizzaWithCheese`, `PizzaWithMushroom`, `PizzaWithCheeseAndMushroom`... jo ki ek nightmare hai).

Decorator Pattern aapko in toppings ko aapas mein "wrap" karne ki permission deta hai.

Stock Market App Example: Notification System

Maan lo aapki stock app mein user ko price update dena hai.

- Default: **Email** notification.
- User adds: **SMS** notification.
- User adds: **WhatsApp** notification.

1. The Component (Interface)

Java

```
interface Notifier {
    void send(String message);
}
```

2. Concrete Component (Basic Class)

Java

```
class EmailNotifier implements Notifier {
    public void send(String message) {
        System.out.println("Sending Email: " + message);
    }
}
```

3. Base Decorator

Yeh class `Notifier` ko implement bhi karegi aur ek `Notifier` object ko apne paas hold bhi karegi.

Java

```java
abstract class NotifierDecorator implements Notifier {
    protected Notifier wrappedNotifier;

    public NotifierDecorator(Notifier notifier) {
        this.wrappedNotifier = notifier;
    }

    public void send(String message) {
        wrappedNotifier.send(message);
    }
}
```

4. Concrete Decorators (Extra Features)

Java

```java
class SMSDecorator extends NotifierDecorator {
    public SMSDecorator(Notifier notifier) { super(notifier); }

    public void send(String message) {
        super.send(message); // Purana (Email) chalne do
        System.out.println("Sending SMS: " + message); // Naya feature
    }
}

class WhatsAppDecorator extends NotifierDecorator {
    public WhatsAppDecorator(Notifier notifier) { super(notifier); }

    public void send(String message) {
        super.send(message);
        System.out.println("Sending WhatsApp message: " + message);
    }
}
```

5. Client Code (Wrapping in Action)

Ab dekho kaise hum runtime par features ko layer-by-layer add karte hain.

Java

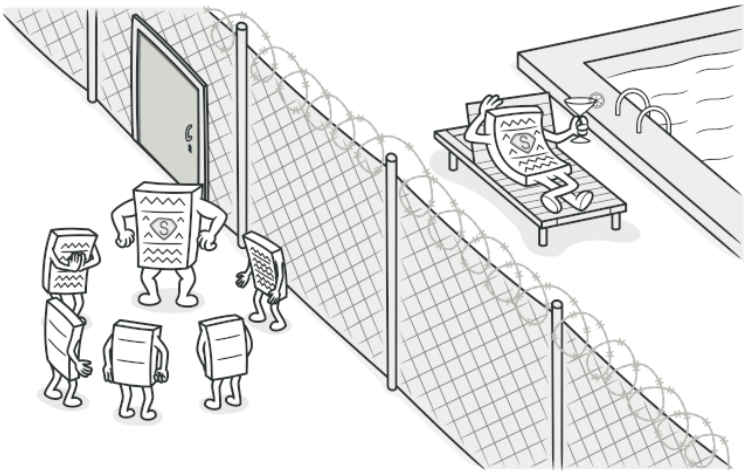
```java
public class Main {
    public static void main(String[] args) {
        // 1. Sirf Email chahiye
        Notifier basic = new EmailNotifier();

        // 2. Email + SMS chahiye
        Notifier withSMS = new SMSDecorator(basic);

        // 3. Email + SMS + WhatsApp teeno chahiye!
        Notifier fullPackage = new WhatsAppDecorator(withSMS);

        System.out.println("Sending combined notification:");
        fullPackage.send("Stock price of Apple hit $200!");
    }
}
```

4. Proxy Pattern:

Proxy is a structural design pattern that lets you provide a substitute or placeholder for another object. A proxy controls access to the original object, allowing you to perform something either before or after the request gets through to the original object.

Proxy Pattern ek aisa Structural Design Pattern hai jo ek "placeholder" ya "representative" object provide karta hai.

Simple words mein: Proxy ek **Security Guard** ya **Middleman** ki tarah kaam karta hai. Jab aap kisi asli object ko direct access nahi dena chahte, toh aap beech mein Proxy laga dete hain.



Proxy Pattern Kyun Use Karte Hain?

Maan lo aapko ek bahut badi image load karni hai ya database se heavy data nikaalna hai. Agar aap program start hote hi ye sab kar denge, toh app slow ho jayegi. Proxy kya karta hai:

1. Asli object tabhi banata hai jab uski **asli zaroorat** ho (**Lazy Loading**).
2. Check karta hai ki user ke paas **permission** hai ya nahi (**Access Control**).

Real-World Example: Internet Proxy

Aapke college ya office mein ek Proxy Server hota hai. Jab aap kisi website (jaise `facebook.com`) ko open karne ki koshish karte ho, toh request pehle Proxy ke paas jati hai.

- Agar wo website blocked hai, toh Proxy aapko mana kar deta hai.
- Agar allowed hai, toh Proxy khud request bhejta hai aur data aapko la kar deta hai.

Java Code Example: Stock Market App (Image Loader)

Socho aapki stock app mein har company ka ek heavy detailed logo hai. Hum nahi chahte ki saare logo ek saath load ho kar memory bhar dein.

Java

```java
interface Image {
    void display();
}
```

2. The Real Object (Heavy Process)

Java

```java
class RealImage implements Image {
    private String fileName;

    public RealImage(String fileName) {
        this.fileName = fileName;
        loadFromDisk(); // Ye bahut heavy operation hai
    }

    private void loadFromDisk() {
        System.out.println("Loading heavy image: " + fileName);
    }

    public void display() {
        System.out.println("Displaying " + fileName);
    }
}
```

3. The Proxy Object

Java

```java
class ProxyImage implements Image {
    private RealImage realImage;
    private String fileName;

    public ProxyImage(String fileName) {
        this.fileName = fileName;
    }

    @Override
    public void display() {
        // Sirf tabhi load karo jab display() call ho
        if (realImage == null) {
            realImage = new RealImage(fileName);
        }
        realImage.display();
    }
}
```

4. Client Code

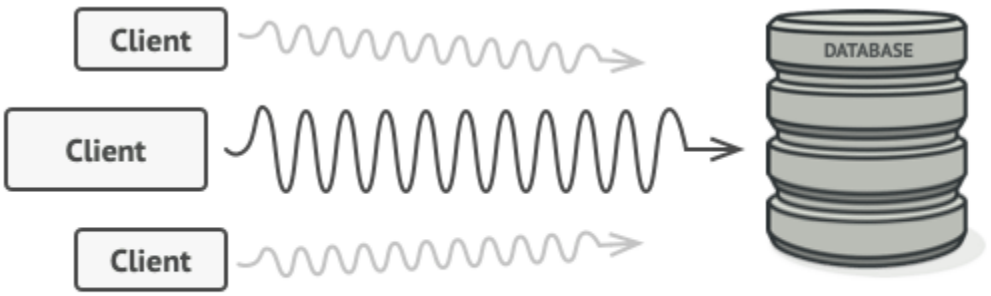
Java

```
public class Main {
    public static void main(String[] args) {
        // Image sirf "setup" hui hai, load nahi hui
        Image stockLogo = new ProxyImage("Reliance_Big_Logo.png");

        // Jab user actually click karega, tabhi load hogi
        System.out.println("User clicks on show logo...");
        stockLogo.display();
    }
}
```

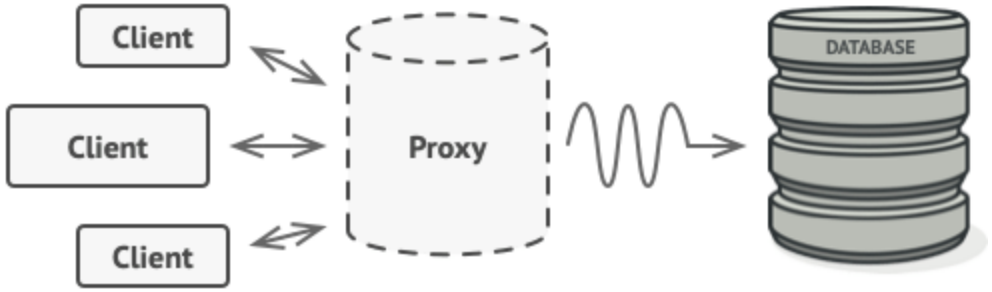
Types of Proxy (Hinglish Summary)

1. **Virtual Proxy:** Heavy objects ko tabhi load karta hai jab unki zaroorat ho (jaise upar wala example).
2. **Protection Proxy:** Security check karta hai (e.g., "Kya ye user Admin hai? Agar nahi, toh delete button mat chalao").
3. **Remote Proxy:** Jo object kisi doosre computer ya server par hai, usse aise dikhata hai jaise wo local machine par hi ho.

Why would you want to control access to an object? Here is an example: you have a massive object that consumes a vast amount of system resources. You need it from time to time, but not always.



Database queries can be really slow.

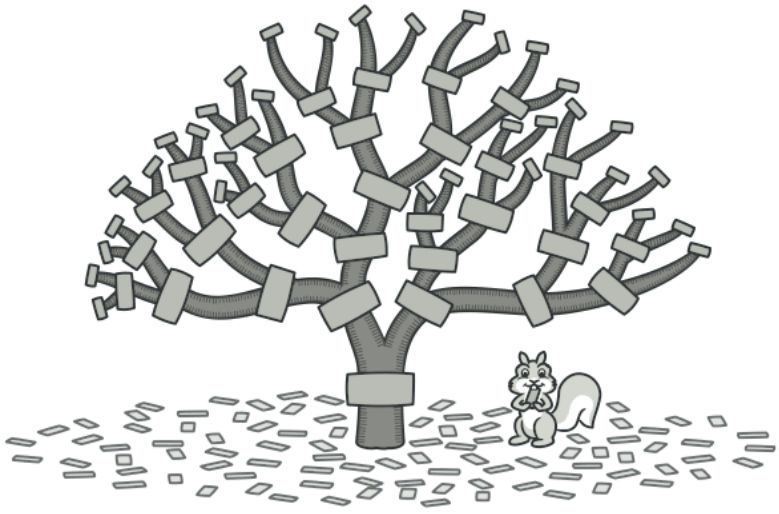


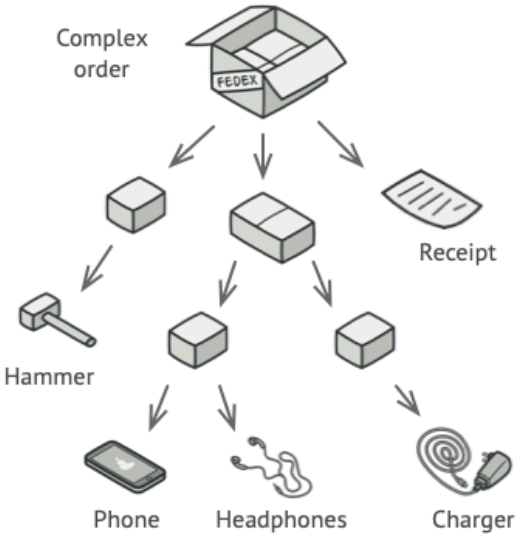
5. Composite Pattern:

Composite is a structural design pattern that lets you compose objects into tree structures and then work with these structures as if they were individual objects.

Composite Pattern ek aisa Structural Design Pattern hai jo aapko objects ko **tree structure** mein organize karne ki permission deta hai taki aap poore structure ko (chahe wo ek single object ho ya group of objects) ek hi tarah se treat kar sakein.

Simple word mein: Jab aapka "Part" (Single item) aur "Whole" (Collection) dono ka behavior same ho, tab hum Composite Pattern use karte hain.





Real-World Example: File System

Socho aapka Computer:

1. Ek **File** (Leaf) hoti hai.
2. Ek **Folder** (Composite) hota hai, jisme Files bhi ho sakti hain aur dusre Folders bhi.

Agar aapko "Size" pata karna hai, toh aap File ka size direct pooch sakte ho. Par Folder ka size uske andar ki saari Files ka sum hota hai. Composite Pattern humein ye flexibility deta hai ki client ko ye na sochna pade ki wo Folder ke saath deal kar raha hai ya File ke saath.

Java Example: Stock Portfolio

Maan lo aapke paas ek Stock App hai:

- Aapke paas kuch **Single Stocks** hain (e.g., Apple, Google).
- Aapke paas kuch **Stock Groups** (Portfolios) hain (e.g., "Tech Stocks", "Blue Chip Stocks").
- Aapko poore group ki value ya single stock ki value nikalni hai.

1. The Component (Interface)

Dono (Stock aur Portfolio) ke liye ek common interface.

Java

```java
interface PortfolioItem {
    void showDetails();
    double getPrice();
}
```

2. The Leaf (Single Stock)

Java

```
class Stock implements PortfolioItem {
    private String name;
    private double price;

    public Stock(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public void showDetails() { System.out.println("Stock: " + name + " | Price:
    public double getPrice() { return price; }
}
```

3. The Composite (Stock Group/Portfolio)

Ye apne andar dusre items (Stocks ya Folders) ko store kar sakta hai.

Java

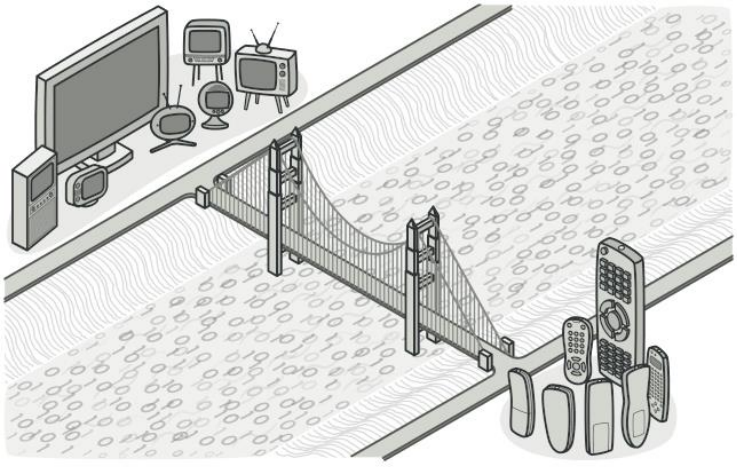
```
import java.util.ArrayList;
import java.util.List;

class StockGroup implements PortfolioItem {
    private String groupName;
    private List<PortfolioItem> items = new ArrayList<>();

    public StockGroup(String name) { this.groupName = name; }

    public void add(PortfolioItem item) { items.add(item); }
```

```
    public void showDetails() {
        System.out.println("\nGroup: " + groupName);
        for (PortfolioItem item : items) {
            item.showDetails(); // Recursion ka use hota hai yaha
        }
    }

    public double getPrice() {
        double total = 0;
        for (PortfolioItem item : items) {
            total += item.getPrice();
        }
        return total;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        PortfolioItem s1 = new Stock("Apple", 150);
        PortfolioItem s2 = new Stock("Google", 2800);

        StockGroup techPortfolio = new StockGroup("Tech Portfolio");
        techPortfolio.add(s1);
        techPortfolio.add(s2);

        PortfolioItem s3 = new Stock("Reliance", 2500);
        StockGroup mainPortfolio = new StockGroup("My Entire Portfolio");
        mainPortfolio.add(techPortfolio); // Group ke andar Group!
        mainPortfolio.add(s3);

        mainPortfolio.showDetails();
        System.out.println("\nTotal Portfolio Value: " + mainPortfolio.getPrice()
    }
}
```

6. Bridge Pattern:

Bridge is a structural design pattern that lets you split a large class or a set of closely related classes into two separate hierarchies—abstraction and implementation—which can be developed independently of each other.



Bridge Pattern ek aisa Structural Design Pattern hai jo **Abstraction** (logic) aur **Implementation** (platform) ko alag-alag kar deta hai taki dono ko ek doosre se independent badhaya (expand kiya) ja sake.

Simple words mein: Agar aapke paas 2 cheezein hain jo badal sakti hain (variation), toh unhe Inheritance (is-a) se connect karne ke bajaye **Composition (has-a)** se connect karo.

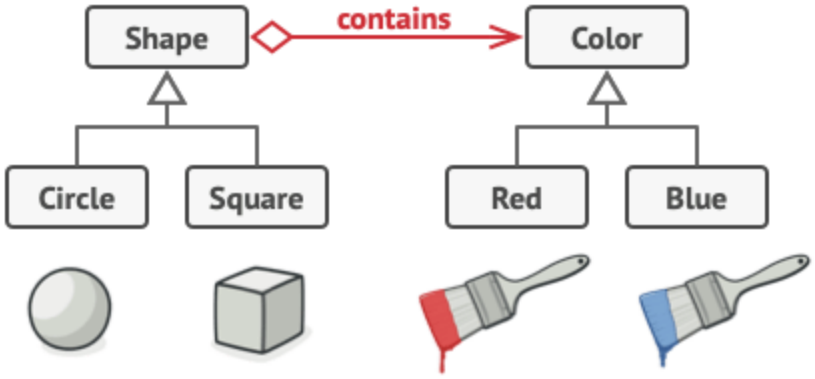
Bridge Pattern Kyun? (The Problem)

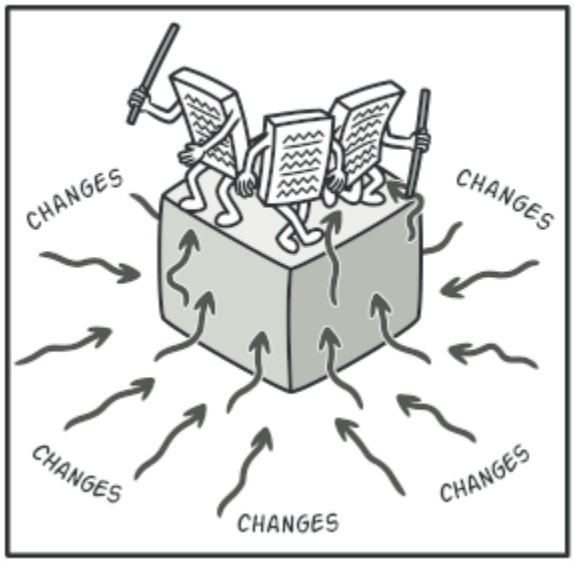
Maan lo aapke paas 2 shapes hain: **Circle** aur **Square**. Aur aapke paas 2 colors hain: **Red** aur **Blue**. Agar aap Inheritance use karoge, toh aapko 4 classes banani padengi:

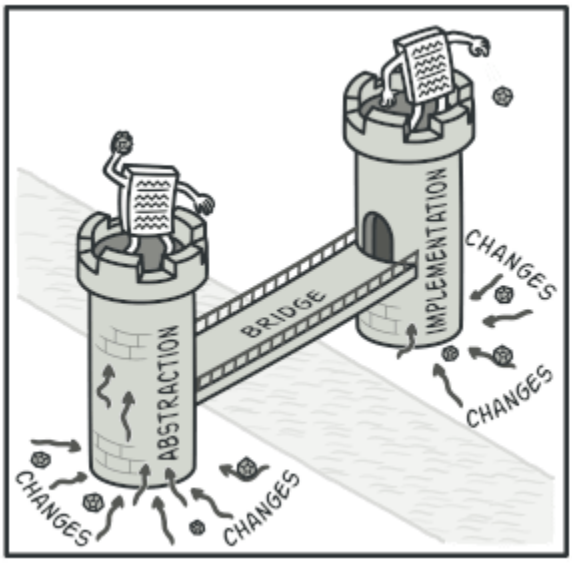
1. `RedCircle`
2. `BlueCircle`
3. `RedSquare`
4. `BlueSquare`

Ab socho agar ek naya shape **Triangle** aur ek naya color **Green** aa jaye, toh classes ka number bahut tezi se badh jayega (Class Explosion).

Bridge Pattern in dono hierarchies ko beech se tod deta hai.


Shape
contains
Color
Circle
Square
Red
Blue


CHANGES
CHANGES
CHANGES
CHANGES
CHANGES
CHANGES


ABSTRACTION
BRIDGE
IMPLEMENTATION
CHANGES
CHANGES
CHANGES
CHANGES
CHANGES

Solution (With Bridge):

Hum Shapes (Abstraction) aur Colors (Implementation) ko do alag hierarchies mein tod dete hain aur unke beech ek **Bridge** bana dete hain.

1. Implementation (The Color Interface)

Pehle hum color ka alag se ek interface banayenge.

Java

```java
interface Color {
    void applyColor();
}

class RedColor implements Color {
    public void applyColor() { System.out.println("Applying Red Color"); }
}

class BlueColor implements Color {
    public void applyColor() { System.out.println("Applying Blue Color"); }
}
```

2. Abstraction (The Shape Class)

Isme hum color ka ek reference rakhte hain. Yahi reference hamara **Bridge** hai.

Java

```java
abstract class Shape {
    // Composition (Bridge)
    protected Color color;

    protected Shape(Color color) {
        this.color = color;
    }

    abstract void draw();
}
```

3. Refined Abstractions (Specific Shapes)

Ab shapes ko color ke internal logic se koi matlab nahi hai.

Java

```
class Circle extends Shape {
    public Circle(Color color) { super(color); }

    public void draw() {
        System.out.print("Drawing Circle. ");
        color.applyColor();
    }
}

class Square extends Shape {
    public Square(Color color) { super(color); }

    public void draw() {
        System.out.print("Drawing Square. ");
        color.applyColor();
    }
}
```

4. Client Code

Ab dekho "Mix and Match" karna kitna aasan hai.

Java

```
public class Main {
    public static void main(String[] args) {
        // Red Circle chahiye?
        Shape redCircle = new Circle(new RedColor());
        redCircle.draw();

        // Blue Square chahiye?
        Shape blueSquare = new Square(new BlueColor());
        blueSquare.draw();
    }
}
```

Iska Fayda Kya Hua? (The Math)

- **Inheritance (Without Bridge):** Agar M shapes hain aur N colors, toh aapko $M \times N$ classes banani padengi.
- **Bridge Pattern:** Aapko sirf $M + N$ classes banani padengi.

Example: Agar aapke paas **5 Shapes** aur **5 Colors** hain:

- Bina Bridge ke: **25 Classes** (Bahut complex!)
- Bridge ke saath: **10 Classes** (Simple aur Flexible)

Behavioral Design Patterns:

Behavioral design patterns are concerned with algorithms and the assignment of responsibilities between objects.

Yeh patterns is baate par focus karte hain ki **objects aapas mein baat (communication) kaise karte hain** aur unki responsibilities kaise divide hoti hain.

1. Observer Pattern:

- **Concept:** Jab ek object (Subject) ki state badalti hai, toh usse jude saare objects (Observers) ko **automatic notification** mil jata hai.
- **Example:** Stock market app mein jaise hi price badalta hai, saare users ko notification chala jata hai.

2. Strategy Pattern:

- **Concept:** Yeh aapko alag-alag algorithms ko classes mein wrap karne deta hai aur aap **runtime par algorithm badal** sakte hain.
- **Example:** Payment page par user choose kar sakta hai ki use UPI, Card, ya NetBanking se pay karna hai.

3. Command Pattern:

- **Concept:** Yeh kisi request ya action ko ek **object ke roop mein convert** kar deta hai, jise aap queue mein daal sakte hain ya undo kar sakte hain.
- **Example:** Remote control ka button dabana; har button ek command object hai jo TV ko signal bhejta hai.

4. State Pattern:

- **Concept:** Yeh kisi object ko apna **behavior badalne** ki permission deta hai jab uski internal state badalti hai (jaise wo class hi badal gayi ho).
- **Example:** Vending machine ki state (No Money -> Has Money -> Out of Stock). Har state mein machine alag behave karegi.

5. Iterator Pattern:

- **Concept:** Yeh kisi collection (list, set) ke elements ko **one-by-one access** karne ka tareeka deta hai bina uske internal structure ko dikhaye.
- **Example:** Java ka `Iterator` jo hum list par loop chalane ke liye use karte hain.

6. Template Method Pattern:

- **Concept:** Yeh ek algorithm ka **skeleton (dhancha)** define karta hai aur sub-classes ko allow karta hai ki wo kuch steps ko override karein bina structure badle.
- **Example:** Tea aur Coffee banane ka process same hai (Boil water -> Add ingredients -> Pour), bas ingredients badal jaate hain.